Ayudantía: TreeSet, TreeMap y PriorityQueue

Colecciones Ordenadas, Algoritmos y Estructuras en Java

Definición en Java: `java.util`

Tres colecciones ordenadas clave del Java Collections Framework:

- `TreeSet<E>`: Conjunto ordenado y sin elementos duplicados.
  - Implementa: `NavigableSet<E>` → `SortedSet<E>` → `Set<E>`.
  - Respaldo interno: Delega en un `TreeMap<E, Object>`.
- `TreeMap<K,V>`: Mapa ordenado por clave ($K \to V$), sin claves duplicadas.
  - Implementa: `NavigableMap<K,V>` → `SortedMap<K,V>` → `Map<K,V>`.
  - Respaldo interno: Árbol Binario de Búsqueda Rojo-Negro (Red-Black Tree).
- `PriorityQueue<E>`: Cola de prioridad basada en un Heap Binario.
  - Implementa: `Queue<E>` → `Collection<E>`.
  - Respaldo interno: Arreglo contiguo dinámico (`Object[]`).

Criterio de Orden: Requieren elementos con `Comparable<T>` (orden natural) o un `Comparator<T>` en el constructor.

TreeSet<E>: Funciones Principales y Uso

Métodos fundamentales:

- **Inserción**: `add(e)` ($O(\log n)$) $\rightarrow$ retorna `false` si ya existía.
- **Eliminación**: `remove(o)`, `pollFirst()`, `pollLast()` ($O(\log n)$).
- **Búsqueda**: `contains(o)` ($O(\log n)$), `first()`, `last()` ($O(1)$ amort.).
- **Rango y cotas**: `ceiling(e)` ($\geq$), `floor(e)` ($\leq$), `higher(e)` ($>$), `lower(e)` ($<$), `subSet(from, to)`.

```
TreeSet<Integer> ts = new TreeSet<>();
ts.add(30); ts.add(10); ts.add(50); ts.add(10); // [10, 30,
    50] (sin dups)

boolean existe = ts.contains(30);  // true - O(log n)
int primero = ts.first();           // 10
int cotaSup = ts.ceiling(25);       // 30 (el menor
    elemento >= 25)
int cotaInf = ts.floor(25);         // 10 (el mayor
    elemento <= 25)
ts.remove(30);                      // elimina 30 en O(log
    n)
```

TreeMap<K,V>: Funciones Principales y Uso

Métodos fundamentales:

- **Inserción / Actualización**: `put(k, v)` ($O(\log n)$).
- **Eliminación**: `remove(k)` ($O(\log n)$), `pollFirstEntry()`, `pollLastEntry()`.
- **Búsqueda**: `get(k)`, `containsKey(k)` ($O(\log n)$), `containsValue(v)` ($O(n)$).
- **Navegación**: `firstKey()`, `lastKey()`, `floorKey(k)`, `ceilingKey(k)`, `subMap(from, to)`, `keySet()`, `values()`.

```
TreeMap<String, Integer> notas = new TreeMap<>();
notas.put("Carlos", 85);
notas.put("Ana", 95);
notas.put("Beatriz", 90);

int notaAna = notas.get("Ana");              // 95 - O(log n)
String primerAlumno = notas.firstKey();     // "Ana"
    (alfabetico)
String antesDeB = notas.lowerKey("Beatriz"); // "Ana" (<
    "Beatriz")
notas.remove("Carlos");                     // elimina en
    O(log n)
```

PriorityQueue<E>: Funciones Principales y Uso

Métodos fundamentales:

- **Inserción**: `offer(e)` / `add(e)` ($O(\log n)$).
- **Eliminar tope**: `poll()` ($O(\log n)$, extrae el mínimo; retorna `null` si vacía).
- **Consultar tope**: `peek()` ($O(1)$, inspecciona sin extraer).
- **Búsqueda general**: `contains(o)` ($O(n)$ lineal, escaneo completo).
- **Orden**: Min-Heap por defecto. Para Max-Heap usar `Comparator.reverseOrder()`.

```
// Min-Heap por defecto (sale el menor primero)
PriorityQueue<Integer> minPQ = new PriorityQueue<>();
minPQ.offer(40); minPQ.offer(10); minPQ.offer(25);
int menor = minPQ.peek(); // 10 en O(1)
int sale = minPQ.poll();  // 10 extraido en O(log n)

// Max-Heap usando Comparator (sale el mayor primero)
PriorityQueue<Integer> maxPQ =
    new PriorityQueue<>(Comparator.reverseOrder());
maxPQ.offer(40); maxPQ.offer(10);
int mayor = maxPQ.poll(); // 40 en O(log n)
```

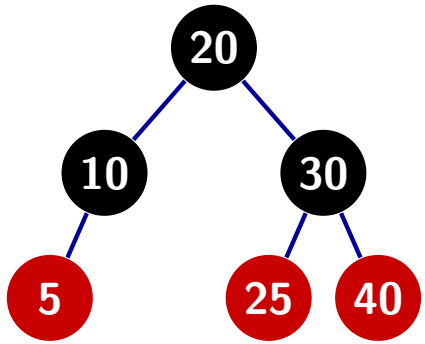
Comparación de Propiedades y Complejidades

Característica	`TreeSet<E>`	`TreeMap<K,V>`	`PriorityQueue<E>`
Inserción	$O(\log n)$	$O(\log n)$	$O(\log n)$
Eliminar (tope)	$O(\log n)$	$O(\log n)$	$O(\log n)$
Eliminar (arbitrario)	$O(\log n)$	$O(\log n)$	$O(n)$
Búsqueda / contains	$O(\log n)$	$O(\log n)$	$O(n)$
Acceso a Extremo	$O(1)$ amort.	$O(1)$ amort.	$O(1)$ (`peek`)
Estructura Base	Árbol R-N	Árbol R-N	Heap en Arreglo
Permite Duplicados	**NO**	Claves **NO**	**SÍ**
Permite `null`	**NO**	Claves **NO**	**NO** (lanza NPE)
Tipo de Orden	Total (Inorden)	Total (Claves)	Parcial (Padre $\leq$ Hijo)

Nota: `TreeSet` y `TreeMap` garantizan orden total bidireccional en todo momento. `PriorityQueue` provee orden parcial para acceso $O(1)$ al elemento prioritario.

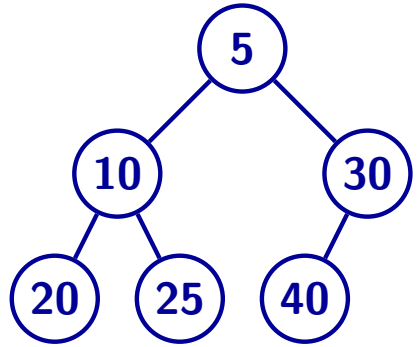
Estructura Interna: Árbol Rojo-Negro vs Heap

TreeSet / TreeMap
Árbol Rojo-Negro (BST)



- BST auto-balanceado con punteros.
- Altura acotada: $h \leq 2\log_2(n+1)$.
- Permite recorrido ordenado inorden.

PriorityQueue
Heap Binario en Arreglo



- Árbol casi completo empaquetado.
- Sin punteros: relaciones aritméticas.
- Máxima localidad de caché y memoria contigua.

Árbol Rojo-Negro: Inserción y Rebalanceo en Palabras

Reglas: (1) Nodos rojos o negros, (2) Raíz siempre negra, (3) Hojas nulas negras, (4) No dos rojos seguidos, (5) Misma altura negra en todo camino raíz-hoja.

Algoritmo de Inserción:

1. **Descenso BST**: Se busca la posición de la clave y se inserta como **ROJO** (preserva la altura negra).
2. **Si el padre es Negro**: Árbol válido. Terminar.
3. **Si el padre es Rojo** (conflicto rojo-rojo), se examina el **tío**:
   - **Caso 1: Tío ROJO (Recoloreo)**: Padre y tío pasan a **NEGRO**, abuelo a **ROJO**. El chequeo sube al abuelo.
   - **Caso 2: Tío NEGRO (Zig-Zag / interior)**: Rotación simple en el padre para alinear en línea recta.
   - **Caso 3: Tío NEGRO (Línea / exterior)**: Rotación en el abuelo en sentido contrario, abuelo pasa a **ROJO** y padre a **NEGRO**. ¡Balanceado!
4. Al finalizar, la raíz siempre se fija a **NEGRO**.

Árbol Rojo-Negro: Eliminación y Rebalanceo en Palabras

Algoritmo de Eliminación:

1. **Búsqueda BST**: Si tiene 2 hijos, se reemplaza con su **sucesor inorden** y se reduce a eliminar un nodo con ≤ 1 hijo.
2. **Caso Simple**: Si el nodo o su reemplazo es **ROJO**, se elimina sin alterar la altura negra.
3. **Doble Negro**: Si se elimina un nodo **NEGRO** y su reemplazo es negro, surge un déficit de color negro en esa rama.

Rebalanceo del Doble Negro (según el hermano):

- **Caso 1 (Hermano Rojo)**: Rotar sobre el padre, swap de color padre-hermano. El nuevo hermano ahora es negro.
- **Caso 2 (Hermano Negro y sobrinos negros)**: Hermano pasa a **ROJO**; el doble negro sube al padre. Si el padre era rojo, queda negro y fin.
- **Caso 3 (Hermano Negro, sobrino cercano rojo)**: Rotar sobre hermano hacia afuera; swap de colores. Pasa directo a Caso 4.
- **Caso 4 (Hermano Negro, sobrino lejano rojo)**: Rotar sobre el padre; hermano toma color del padre, padre y sobrino quedan **NEGROS**. ¡Resuelto!

PriorityQueue: Matemática de Índices y Niveles

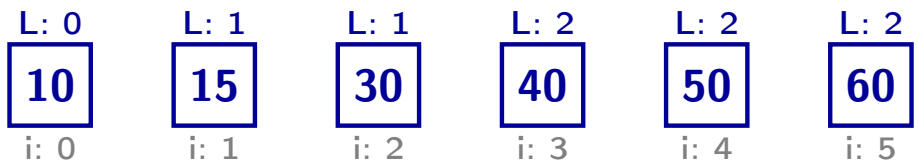
Representación en Arreglo 0-indexado: Árbol binario casi completo empaquetado secuencialmente.

Fórmulas de Navegación:

- **Padre de i**: $\text{padre}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor \quad (i > 0)$
- **Hijo Izquierdo**: $\text{izq}(i) = 2i + 1$
- **Hijo Derecho**: $\text{der}(i) = 2i + 2$
- **Hojas**: índices $\lfloor n/2 \rfloor$ hasta $n - 1$.

Fórmulas de Niveles y Altura:

- **Nivel de i**: $L = \lfloor \log_2(i + 1) \rfloor$
- **Rango nivel L**: $[2^L - 1,\ 2^{L+1} - 2]$
- **Altura total**: $h = \lfloor \log_2 n \rfloor$
- **Internos**: índices 0 a $\lfloor n/2 \rfloor - 1$.



PriorityQueue: Inserción (`siftUp`) en Palabras

Operación `offer(x)` / `add(x)`: Inserta un elemento manteniendo la forma de árbol casi completo y la propiedad de orden de heap.

Algoritmo paso a paso:

1. **Inserción al final**: Se coloca el nuevo elemento x en la primera posición disponible del arreglo: índice $i = n$. Se incrementa el tamaño n.
2. **Flotar (`siftUp`)**:
   - Mientras el nodo no sea la raíz ($i > 0$):
   - Se calcula el índice del padre: $p = \lfloor (i-1)/2 \rfloor$.
   - **Comparación**: Si $x <$ arreglo[p] (en min-heap):
     - Se intercambia x con su padre arreglo[p].
     - El índice activo pasa a ser $i = p$ y se repite hacia arriba.
   - Si $x \geq$ arreglo[p]: Condición de heap cumplida. **Fin**.
3. Si se llega a la raíz ($i = 0$): Nuevo mínimo global. **Fin**.

Complejidad: Peor caso recorre la altura del árbol $\Rightarrow O(\log n)$ comparaciones y swaps.

PriorityQueue: Eliminación (`siftDown`) y `heapify`

Extracción del tope (`poll()`):

1. Se guarda el valor de la raíz (arreglo[0], el mínimo).
2. Se mueve el **último elemento** (arreglo[$n-1$]) a la raíz ($i = 0$) y se reduce $n \to n-1$.
3. **Hundir (`siftDown`):**
   - Mientras i tenga hijos (izq(i) $= 2i+1 < n$):
   - Identificar el **hijo de menor valor** entre izq(i) y der(i) (si existe).
   - Si el elemento actual es mayor que dicho hijo menor: intercambiarlos y descender ($i \leftarrow$ hijo menor).
   - Si es menor o igual a ambos hijos: propiedad de heap satisfecha. **Fin**.

Construcción Lineal (`heapify`):

- Para armar un heap desde un arreglo desordenado, se aplica `siftDown` desde el último nodo interno $i = \lfloor n/2 \rfloor - 1$ retrocediendo hasta la raíz $i = 0$.
- **Complejidad**: $\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} O(h) = \mathrm{O(n)}$ (óptimo frente a $O(n \log n)$).

Criterio de Selección en Algoritmos y Competencias

¿Cuál estructura elegir según el problema?

`PriorityQueue`

Solo requieres procesar el **mínimo o máximo** dinámicamente.

- Dijkstra / Prim.
- Codificación Huffman.
- Top-k elementos mayores o menores.
- Simulación de eventos por tiempo.

`TreeSet`

Requieres conjunto **sin duplicados** con consultas de vecinos o rangos.

- Barrido geométrico (sweep-line).
- Cota superior/inferior: `ceiling` / `floor`.
- Saber si un valor existe en $O(\log n)$.

`TreeMap`

Requieres asociar **clave** $\rightarrow$ **valor** con orden en las claves.

- Tablas de intervalos por tramos.
- Compresión de coordenadas discretas.
- Histogramas ordenados alfabéticamente.